Lab Assignment 7 – Nearest Location Recommender over a Road Network

API Link: https://nearest-locations.onrender.com/search/

1. Problem

Given 10,000 locations (`ID, Latitude, Longitude, Category`) on a 1 × 1 grid, and a road-linkage file, return the **10 closest locations** of a requested category that lie inside a search radius.

- **Closeness** = actual traversal (road / grid) distance through the linkage graph.
- **Radius filter** = straight-line (circular) distance from the current position.

2. Dataset analysis

- 10,000 points form a regular **100 × 100 grid**; spacing = 1/99 ≈ 0.010101.
- IDs are row-major: `ID = row·100 + col + 1`, with `lat = row/99`, `lon = col/99`.
- 8 categories (bank, cafe, hospital, park, pharmacy, restaurant, school, store) with 1,250 points each.
- Coordinates in the CSV are rounded to 6 decimals, so they are **not exactly symmetric** (e.g. one step can be 0.010101 or 0.010102). This matters for tie-breaking (Section 4.3).

3. API

Live deployment: https://nearest-locations.onrender.com/search/ · Web UI: https://nearest-locations.onrender.com/ui (free Render instance, so the first request after it has been idle can take ~50 s)

Route: `/search/` (GET or POST)

Field	Meaning
`lat`	current latitude
`long`	current longitude
`cat`	category
`rad`	search radius (circular distance)
`link`	road linkage `.txt`: each line `a b` is a road between IDs `a` and `b`

`link` can be sent as an uploaded file (multipart), as raw text, as a URL to the txt file, or as a filename that is on the server. If `link` is missing, the API falls back to a full 4-neighbour grid.

Example:

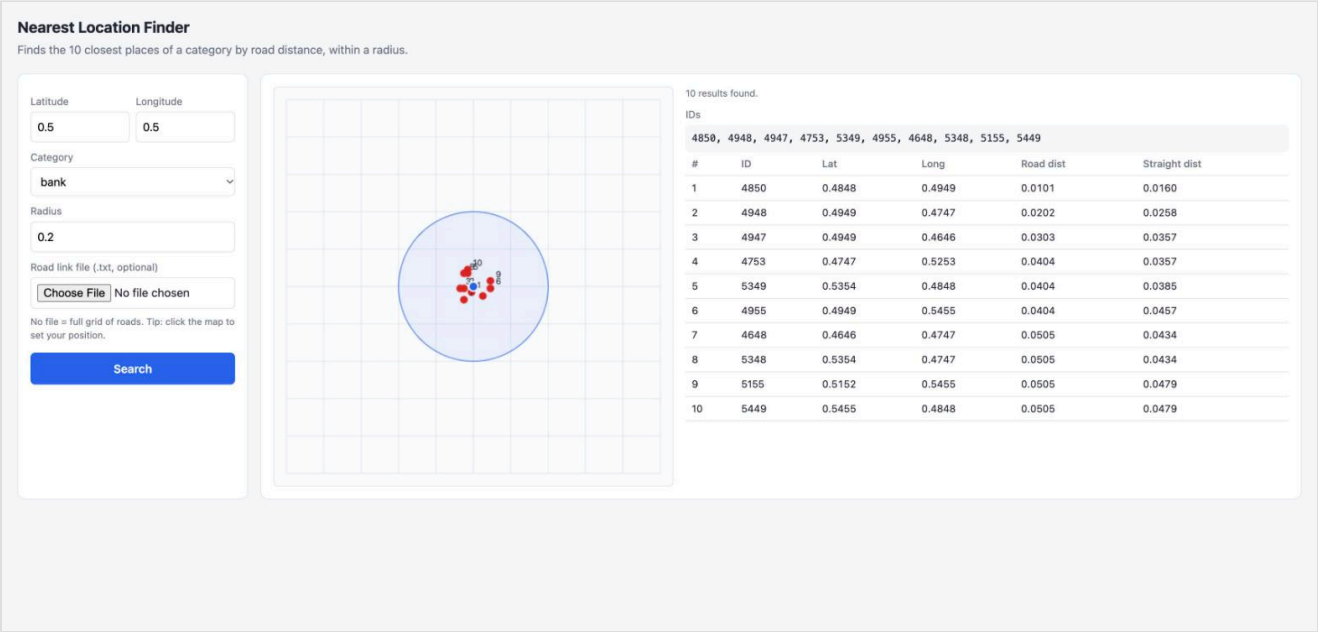
```
curl -X POST https://nearest-locations.onrender.com/search/ -F lat=0.5 -F long=0.5 -F ca
t=bank -F rad=0.2 -F link=@sample_link.txt
```

Response:


```
{"ids": [4850, 4948, 4947, 4753, 5349, 4648, 5348, 5155, 5449, 5452],
 "results": [{"id": 4850, "lat": 0.484848, "long": 0.494949, "category": "bank",
              "grid_distance": 0.010101, "hops": 1.0, "radial_distance": 0.015972, "reac
hable": true}, ...]}
```


Web interface

A small web page is served at **`/ui`**. It has a form for `lat`, `long`, `cat`, `rad`, and an optional link-file upload. It calls `/search/` and shows the 10 results in a table and on a 1 × 1 map with the current position and the search circle. You can click the map to set the current position.



4. Method

4.1 Graph construction

The link file is parsed into an **adjacency list** (undirected). The length of each edge is the Euclidean length of the road segment in **grid units**, which is exactly 1 for neighbouring points. Parsed graphs are **cached** (LRU) by file content, so repeat queries with the same link file skip parsing. 0-based IDs in a link file are detected and shifted automatically.

4.2 Search algorithm

1. **Radius filter (vectorised NumPy):** compute the circular distance from `(lat, long)` to all 10,000 points at once, and keep the points with the requested category and `distance ≤ rad`. These are the *targets*.

2. **Source node:** the grid point nearest to `(lat, long)`. If the query lies exactly between several points (e.g. a cell centre), all tied points are used as sources (multi-source Dijkstra).
3. **Dijkstra with early stopping:** expand nodes in increasing road distance. Each time a target is popped, record it, and stop when 10 targets have been found. Nodes tied at the 10th distance are also drained, so the cut-off is deterministic.
4. **Fallback:** if roads cannot reach 10 targets, the remaining slots are filled with the unreachable targets nearest by straight line (they are flagged `reachable: false`).

4.3 Tie-breaking

Grid networks have many equal path lengths, so ties are broken by **(road distance, straight-line distance, ID)**. Road distances use exact integer grid units, and the straight-line distance is rounded to 5 decimals. This keeps the CSV's 6-decimal rounding noise from reordering locations that are truly tied.

4.4 Why not Euclidean / k-d tree only?

With missing links, a point that is close in a straight line can be far by road, or unreachable. A spatial index can only do the radius filter. The ranking must come from shortest paths in the graph.

5. Complexity

- Graph build: O(E), once per distinct link file (cached).
- Query: O(N) vectorised radius filter, plus Dijkstra O((V' + E') log V'), where V' is only the part of the graph explored before 10 targets are found (early stop). In practice that is ~1–20 ms for 10,000 nodes.

6. Validation

A test link file was generated by randomly deleting 25% of the roads of the 100 × 100 grid. The API output was compared against an independent brute-force BFS (full shortest-path labelling, then filtering and sorting) on 6 queries: on-grid points, off-grid points, corners, cell centres, and various categories and radii. **All 6 matched exactly (60/60 IDs).**

7. Tech stack

Python 3, FastAPI + Uvicorn, NumPy, `heapq` (binary-heap Dijkstra); plain HTML/JS frontend (`static/index.html`). Deployable on Render or Railway (`render.yaml`, `Procfile`, `requirements.txt` included).